



**UNIVERSIDAD TECNOLÓGICA DE LA
HUASTECA HIDALGUENSE**

Organismo Descentralizado de la Administración Pública Estatal

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE



ASIGNATURA: GESTIÓN DEL PROCESO DE DESARROLLO DE SOFTWARE

ACTIVIDAD 2

**DOCUMENTACIÓN DEL PROCESO DE PLANEACIÓN Y CONFIGURACIÓN DEL
REPOSITORIO DEL PROYECTO MÓVIL**

ALUMNOS:

HERNÁNDEZ MEZA ALEXANDER 20230106

RUBIO ÁNGELES PEDRO 20230074

DOCENTE: ING. ANA MARÍA FELIPE REDONDO

CUATRIMESTRE: 10º

GRUPO: "A"

HUEJUTLA DE REYES, HIDALGO

28 DE SEPTIEMBRE DE 2026

Entregables

La actividad pide tres entregables. Los dos primeros son enlaces y el tercero es este documento.

Tabla 1

Entregables de la actividad y ubicación de cada uno

Entregable	Dónde está
Planeación y seguimiento	https://github.com/users/Ennimex/projects/2
Código fuente	https://github.com/Ennimex/pier-reposteria-app
Evidencia de la configuración realizada	Este documento (secciones 1 y 2), publicado también en el repositorio como docs/ACTIVIDAD2_EVIDENCIAS.pdf

El tablero es privado. La docente tiene acceso de lectura con la cuenta **afelipe23**. El repositorio es público.

1. Planeación del proyecto móvil con enfoque DevOps

1.1 Herramienta y forma de trabajo

La planeación se lleva en **GitHub Projects**, en el tablero «Pier Móvil — Planeación DevOps» (GitHub, s.f.-a). Se trabaja con **Scrumban**: sprints de dos semanas para planear y entregar, y un tablero Kanban para el flujo diario con los tres estados predeterminados de GitHub Projects: Todo (por hacer, aún no se empieza), In Progress (en progreso) y Done (hecho).

Los sprints van de miércoles a martes. La entrega y revisión de cada sprint es el martes de cierre, día en que coinciden las clases de Desarrollo Móvil Integral y de Gestión del Proceso de Desarrollo de Software.

Cada tarjeta es un issue del repositorio y lleva estos campos: estado, responsable, sprint, tipo (épica, historia o tarea), plataforma, prioridad, puntos de historia y milestone. El tablero tiene tres vistas: Épicas, con el avance de cada funcionalidad; Tablero, el Kanban por estado, que también se consulta por sprint; y Cronograma, la vista tipo Gantt.

Figura 1

Tablero Kanban con el estado de cada actividad

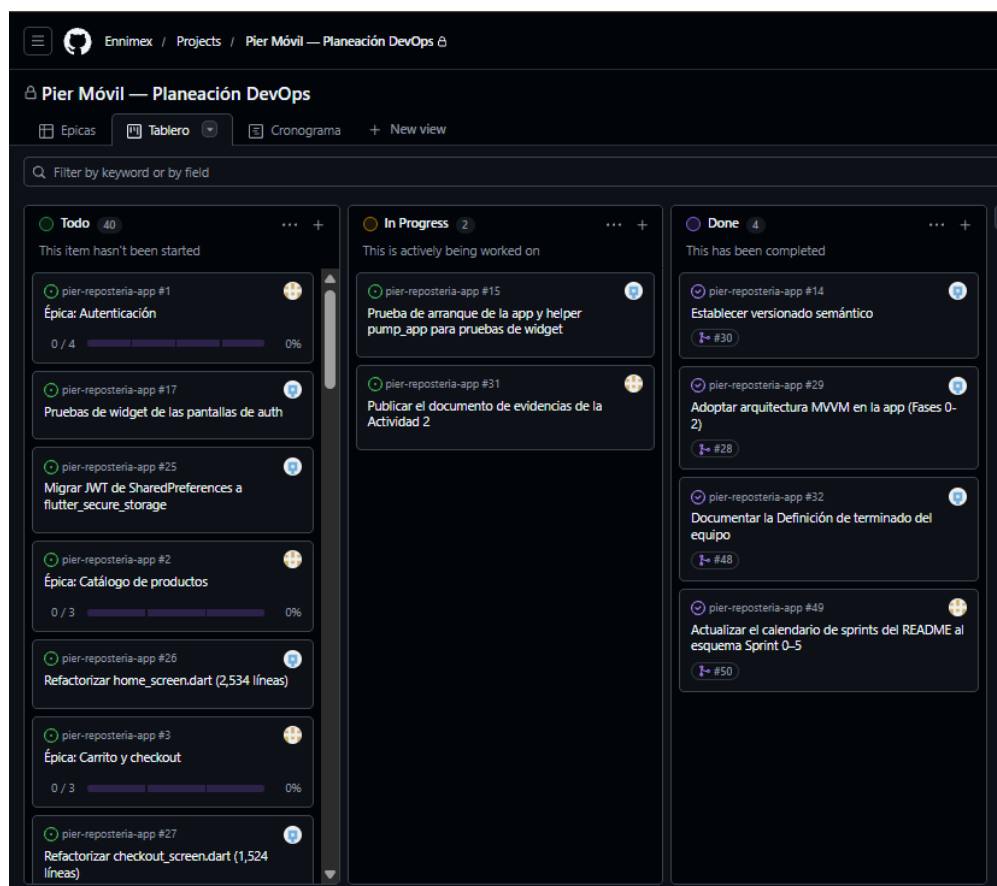
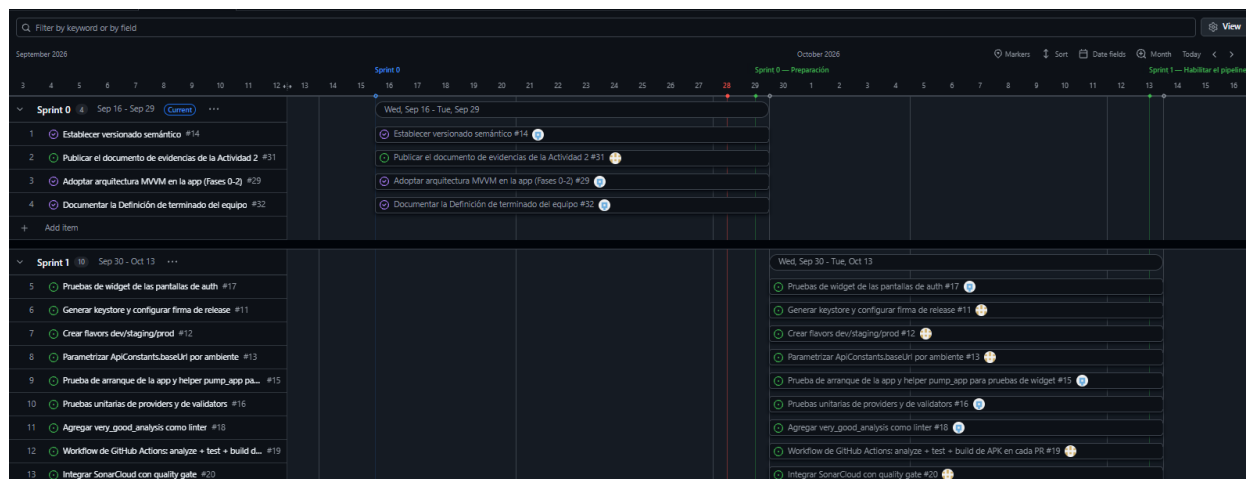


Figura 2

Cronograma tipo Gantt por sprint



1.2 Funcionalidades principales de la aplicación

Cada funcionalidad de la app es una épica del tablero. Las historias y tareas cuelgan de su épica como sub-issues, de modo que el avance de la épica se calcula solo.

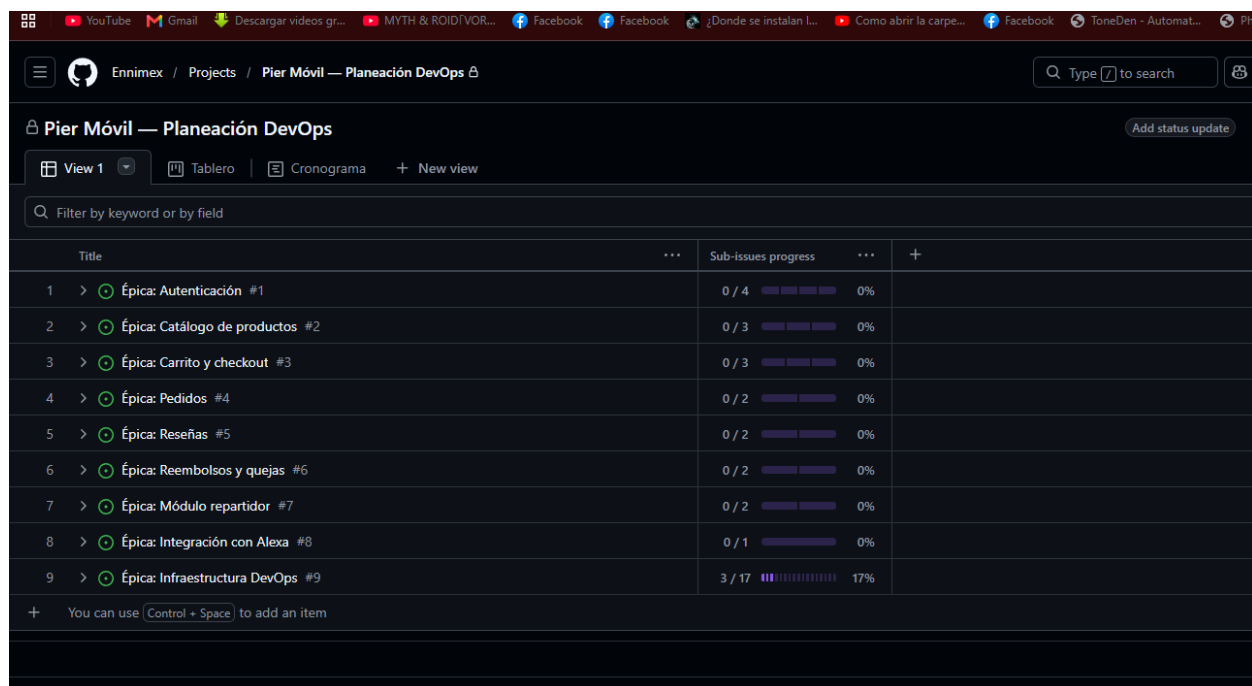
Tabla 2

Funcionalidades principales de la aplicación organizadas como épicas

Épica	Funcionalidad	Qué permite
#1	Autenticación	Registro, inicio de sesión con correo o con Google y verificación de correo.
#2	Catálogo de productos	Explorar y buscar productos, ver su detalle y guardar favoritos.
#3	Carrito y checkout	Armar el carrito, elegir dirección de entrega y pagar con tarjeta.
#4	Pedidos	Consultar el historial y dar seguimiento al estado de cada pedido.
#5	Reseñas	Calificar los productos comprados y consultar las reseñas de otros clientes.
#6	Reembolsos y quejas	Solicitar un reembolso, levantar una queja y ver su seguimiento.
#7	Módulo repartidor	Ver las entregas asignadas y registrar la evidencia de entrega.
#8	Integración con Alexa	Vincular la cuenta para consultar pedidos por voz.
#9	Infraestructura DevOps	Épica técnica: ambientes, firma, versionado, pruebas, CI/CD y monitoreo.

Figura 3

Épicas del proyecto con el avance de sus sub-issues



1.3 Integrantes, roles y responsabilidades

Tabla 3

Integrantes, roles y responsabilidades del equipo

Integrante	Rol en Scrumban	Responsabilidades
Pedro Rubio Angeles (@PedroRubioo)	Product Owner · DevOps / Release Engineer	Define y prioriza las épicas del backlog. Ambientes, firma y publicación de liberaciones, pipeline de CI/CD y monitoreo. Revisa y aprueba los Pull Requests hacia main. Responsable del backend (API) que consume la app.
Alexander Hernandez Meza (@Ennimex)	Desarrollo móvil · QA · Documentación técnica	Implementa las historias de la app en Flutter y la arquitectura MVVM. Pruebas unitarias, de widget y de integración. Versionado, README y documentación técnica.

Esta misma tabla está publicada en la descripción del tablero y en el README del repositorio, para que quien abra cualquiera de los dos enlaces la encuentre.

Figura 4

Descripción del tablero con integrantes, roles y responsabilidades



1.4 Historias de usuario y tareas organizadas por iteraciones

El backlog planeado tiene **37 actividades**: 25 historias de usuario y 12 tareas, repartidas en seis sprints, del Sprint 0 al Sprint 5. Las historias siguen el formato «Como... quiero... para...» y llevan criterios de aceptación en Gherkin. Las tareas llevan objetivo, pasos y criterio de terminado. Ambas se crean desde plantillas del repositorio para que todas tengan la misma estructura. En la columna Estado, Todo significa por hacer (aún no se empieza), In Progress en progreso y Done hecho.

Tabla 4

Actividades del Sprint 0 — Preparación (16-sep a 29-sep)

#	Actividad	Tipo	Responsable	Estado
#14	Establecer versionado semántico	Tarea	Alexander Hernández	Done
#29	Adoptar arquitectura MVVM en la app (Fases 0-2)	Tarea	Alexander Hernández	Done
#31	Publicar el documento de evidencias de la Actividad 2	Tarea	Pedro Rubio	In Progress
#32	Documentar la Definición de terminado del equipo	Tarea	Alexander Hernández	Done
#49	Actualizar el calendario de sprints del README al esquema Sprint 0–5	Tarea	Pedro Rubio	Done

Tabla 5*Actividades del Sprint 1 — Habilitar el pipeline (30-sep a 13-oct)*

#	Actividad	Tipo	Responsable	Estado
#11	Generar keystore y configurar firma de release	Historia	Pedro Rubio	Todo
#12	Crear flavors dev/staging/prod	Historia	Pedro Rubio	Todo
#13	Parametrizar ApiConstants.baseUrl por ambiente	Historia	Pedro Rubio	Todo
#15	Prueba de arranque de la app y helper pump_app para pruebas de widget	Tarea	Alexander Hernández	In Progress
#16	Pruebas unitarias de providers y de validators	Tarea	Alexander Hernández	Todo
#17	Pruebas de widget de las pantallas de auth	Tarea	Alexander Hernández	Todo
#18	Agregar very_good_analysis como linter	Tarea	Alexander Hernández	Todo
#19	Workflow de GitHub Actions: analyze + test + build de APK en cada PR	Historia	Pedro Rubio	Todo
#20	Integrar SonarCloud con quality gate	Historia	Pedro Rubio	Todo
#21	Pruebas de integración en emulador con reactivecircus/android-emulator-runner	Historia	Alexander Hernández	Todo

Tabla 6*Actividades del Sprint 2 — Calidad y pruebas (14-oct a 27-oct)*

#	Actividad	Tipo	Responsable	Estado
#25	Migrar JWT de SharedPreferences a flutter_secure_storage	Tarea	Alexander Hernández	Todo
#26	Refactorizar home_screen.dart (2,534 líneas)	Tarea	Alexander Hernández	Todo
#27	Refactorizar checkout_screen.dart (1,524 líneas)	Tarea	Alexander Hernández	Todo
#33	Registro y verificación de correo con código	Historia	Alexander Hernández	Todo
#34	Inicio de sesión con correo o Google y recuperación de contraseña	Historia	Alexander Hernández	Todo
#35	Carrito: agregar productos y ajustar cantidades	Historia	Alexander Hernández	Todo
#36	Checkout: tipo de entrega y pago con tarjeta	Historia	Alexander Hernández	Todo

Tabla 7*Actividades del Sprint 3 — Liberación y monitoreo (28-oct a 10-nov)*

#	Actividad	Tipo	Responsable	Estado
#10	Cambiar applicationId a mx.com.pierreposteria	Historia	Pedro Rubio	Todo
#22	Publicar el APK como artifact y adjuntarlo a GitHub Releases al hacer tag	Historia	Pedro Rubio	Todo
#23	Integrar sentry_flutter para errores y rendimiento	Historia	Pedro Rubio	Todo
#24	Definir métricas y niveles de servicio con Sentry y Play Console Vitals	Historia	Pedro Rubio	Todo

Tabla 8*Actividades del Sprint 4 — Catálogo, pedidos y reseñas (11-nov a 24-nov)*

#	Actividad	Tipo	Responsable	Estado
#37	Catálogo: búsqueda y filtros de productos	Historia	Alexander Hernández	Todo
#38	Favoritos y «Avísame» en productos agotados	Historia	Alexander Hernández	Todo
#39	Mis pedidos: activos, historial y detalle	Historia	Pedro Rubio	Todo
#40	Cancelar un pedido que aún es cancelable	Historia	Pedro Rubio	Todo
#41	Calificar un producto comprado	Historia	Alexander Hernández	Todo
#42	Consultar reseñas y editar las propias	Historia	Alexander Hernández	Todo

Tabla 9*Actividades del Sprint 5 — Reembolsos, repartidor y Alexa (25-nov a 08-dic)*

#	Actividad	Tipo	Responsable	Estado
#43	Solicitar el reembolso de un pedido completado	Historia	Pedro Rubio	Todo
#44	Enviar una queja, sugerencia o comentario y ver su estado	Historia	Pedro Rubio	Todo
#45	Repartidor: disponibilidad y aceptación de pedidos	Historia	Alexander Hernández	Todo
#46	Repartidor: aviso de llegada y confirmación de entrega	Historia	Alexander Hernández	Todo
#47	Vincular la cuenta con Alexa mediante un código	Historia	Pedro Rubio	Todo

Las historias de los sprints 2, 4 y 5 describen funcionalidad que la app ya tiene. Su trabajo en cada sprint es validarlas: sus criterios de aceptación en Gherkin son la base de las pruebas de widget y de integración que se automatizan a partir del Sprint 1.

Figura 5

Historias y tareas agrupadas por iteración

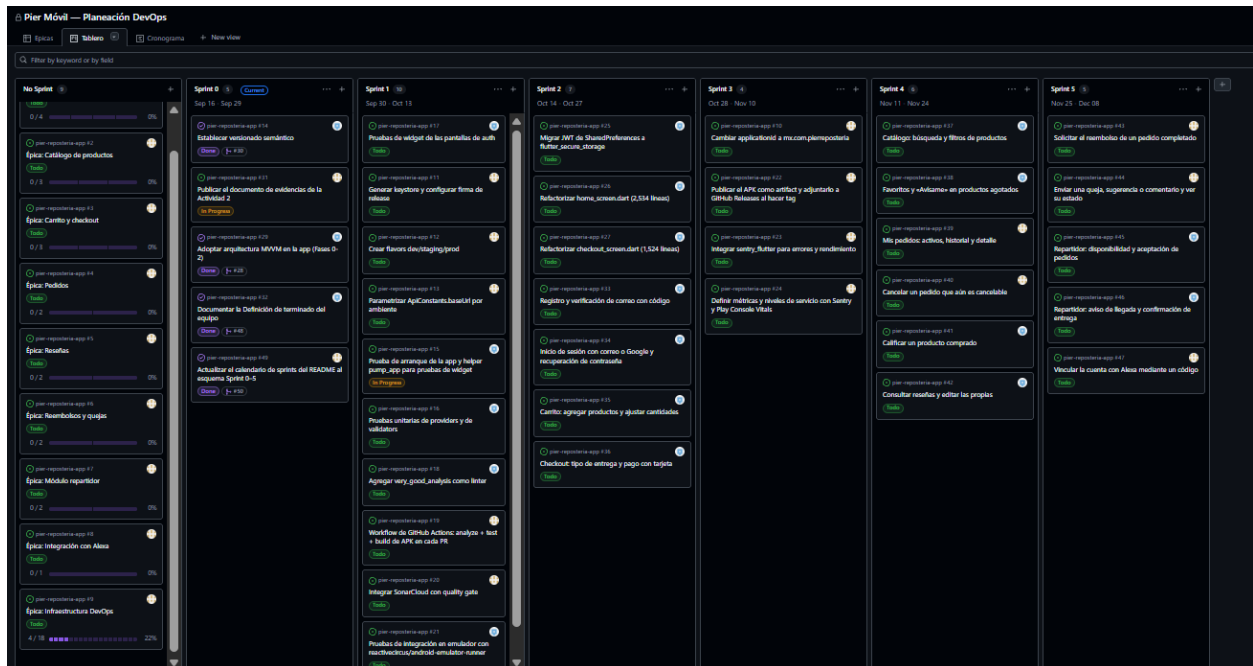


Figura 6

Ejemplo de historia de usuario con criterios de aceptación

Generar keystore y configurar firma de release #11 New issue

Open Parent: Épica: Infraestructura DevOps

Ennimex opened 3d ago Owner

Historia

Como equipo quiero que los APK de release se firmen con una llave propia para poder publicarlos en Google Play y para que las actualizaciones se reconozcan como la misma app.

Contexto

android/app/build.gradle.kts línea 37: signingConfig = signingConfigs.getByname("debug") . Hoy el release se firma con la llave de debug.

Pasos

- Generar el keystore: `keytool -genkey -v -keystore pier-release.jks -keyalg RSA -keysize 2048 -validity 10000 -alias pier`
- Crear `android/key.properties` con `storeFile`, `storePassword`, `keyAlias`, `keyPassword` (ya está en `.gitignore`, igual que `*.jks`)
- En `build.gradle.kts`: leer `key.properties`, declarar `signingConfigs.create("release")` y usarlo en `buildTypes.release`
- Guardar el keystore fuera del repo (y una copia cifrada en un lugar seguro del equipo)
- Para CI: secretos `ANDROID_KEYSTORE_BASE64`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD` en GitHub → el workflow los decodifica en tiempo de build

Criterios de aceptación

Escenario: el APK de release lleva la firma propia
 Dado que existe `key.properties` con el keystore del equipo
 Cuando ejecuto `"Flutter build apk --release"`
 Entonces `"apksigner verify --print-certs"` muestra el certificado del equipo y no "Android Debug"

Escenario: no se filtran credenciales
 Dado que el keystore y `key.properties` existen localmente
 Cuando ejecuto `"git ls-files | grep -E 'jks|keystore|key.properties'"`
 Entonces el resultado está vacío

Plataforma: Móvil · Prioridad: Alta · Épica relacionada: [#9](#)

Assignees
 PedroRubio

Labels
 devops historia seguridad

Projects
 No projects

Milestone
 Sprint 1 — Habilitar el pipeline
 Due by September 28, 2026, 57% complete

Relationships
 Parent issue
 Épica: Infraestructura DevOps
 Ennimex/pier-reposteria-app#9

Development
 No branches or pull requests

Participants
 Ennimex, PedroRubio

[Open in GitHub Copilot app](#)

1.5 Responsable y estado de cada actividad

El responsable es el campo Assignees de cada issue y el estado es la columna Status del tablero: Todo (por hacer), In Progress (en progreso) o Done (hecho). Las tablas de la sección 1.4 muestran ambos para cada actividad. A la fecha de este documento el resumen es:

Tabla 10*Resumen de actividades por estado y por responsable*

Estado	Actividades	Responsable	Actividades
Todo	31	Pedro Rubio	16
In Progress	2	Alexander Hernández	21
Done	4		

El estado no se actualiza a mano al terminar. Cuando un Pull Request que dice «Closes #N» se mezcla a main, GitHub cierra el issue y el tablero mueve la tarjeta a Done. Se comprobó con los issues #14, #29, #32 y #49.

1.6 Producto verificable de cada sprint

Cada sprint es también un milestone del repositorio. La descripción del milestone declara el producto verificable, es decir, lo que se puede demostrar funcionando el martes de cierre.

El calendario arranca con un Sprint 0 de preparación. La primera quincena se había planeado como Sprint 1; el 23 de septiembre se reorganizó para dedicarla a preparar el repositorio, el tablero, el entorno y la arquitectura antes de construir el pipeline, y el trabajo de desarrollo se reacomodó en los Sprints 1 a 5 con el mismo cierre del 8 de diciembre. Los milestones, las iteraciones del tablero y el README se actualizaron al nuevo esquema (issue #49, Pull Request #50). Las figuras 6, 8, 9 y 17 son anteriores a ese cambio, por eso muestran el milestone con el nombre o la fecha de entonces.

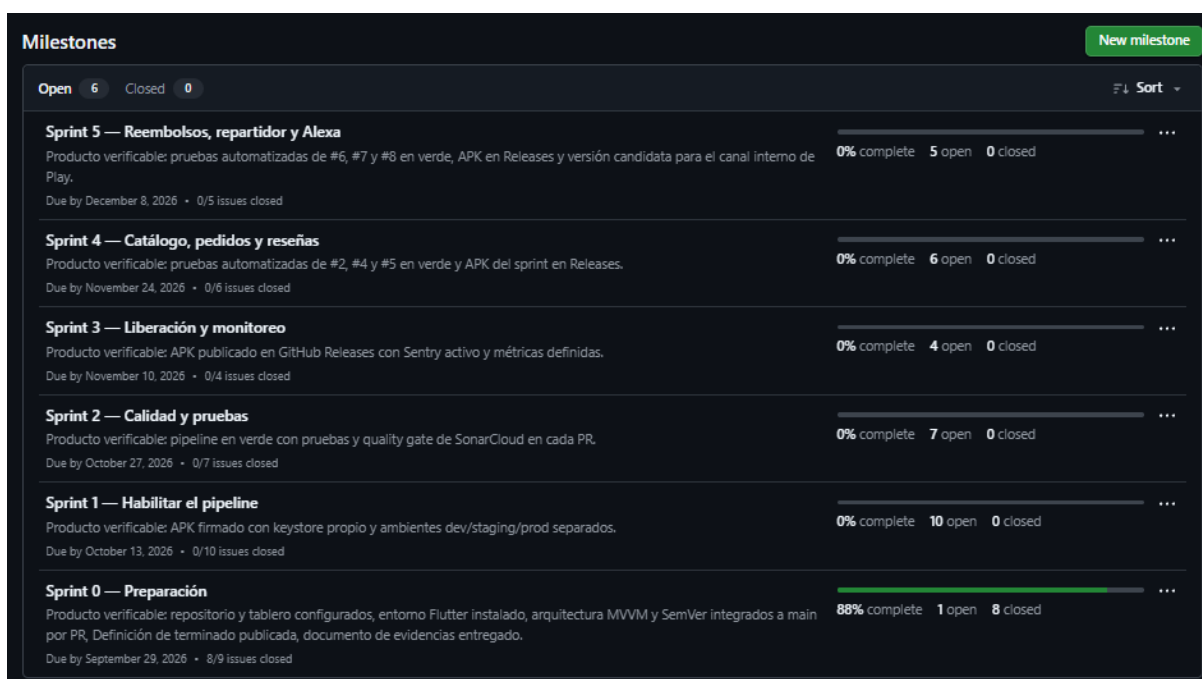
Tabla 11*Producto verificable de cada sprint*

Sprint	Fechas	Producto verificable
0 · Preparación	16-sep a 29-sep	Repositorio y tablero configurados, entorno Flutter instalado, arquitectura MVVM y SemVer integrados a main por Pull Request, Definición de terminado publicada y documento de evidencias entregado.
1 · Habilitar el pipeline	30-sep a 13-oct	APK firmado con keystore propio y ambientes separados (dev, staging y prod).

Sprint	Fechas	Producto verificable
2 · Calidad y pruebas	14-oct a 27-oct	Pipeline en verde con pruebas y quality gate de SonarCloud en cada Pull Request.
3 · Liberación y monitoreo	28-oct a 10-nov	APK publicado en GitHub Releases con Sentry activo y métricas de servicio definidas.
4 · Catálogo, pedidos y reseñas	11-nov a 24-nov	Pruebas automatizadas de las historias de Catálogo (#2), Pedidos (#4) y Reseñas (#5) en verde y APK del sprint en GitHub Releases.
5 · Reembolsos, repartidor y Alexa	25-nov a 08-dic	Pruebas automatizadas de las historias de Reembolsos y quejas (#6), Repartidor (#7) y Alexa (#8) en verde, APK del sprint en GitHub Releases y versión candidata para el canal interno de Google Play.

Figura 7

Milestones del repositorio con el producto verificable de cada sprint



1.7 Relación de las actividades con issues, ramas, Pull Requests y commits

La trazabilidad sigue siempre la misma cadena, y cada eslabón enlaza al siguiente dentro de GitHub:

- **Épica** → **historia o tarea**: la actividad es sub-issue de su épica.
- **Actividad** → **rama**: el trabajo nace en una rama `feature/*` creada desde `main`.

- **Rama** → **commits**: mensajes con Conventional Commits que citan el issue (Refs #N).
- **Commits** → **Pull Request**: la descripción incluye Closes #N, lo que enlaza el PR con el issue.
- **Pull Request** → **main**: revisión del compañero, aprobación y merge. El issue se cierra y la tarjeta pasa a Done.

Tabla 12

Relación de las actividades con ramas, Pull Requests y commits

Actividad	Rama	Pull Request	Commits	Resultado
#29 Adoptar arquitectura MVVM (Fases 0-2)	mvvm	#28	5	Aprobado por PedroRubioo y mezclado el 19-sep. El issue se cerró solo.
#14 Establecer versionado semántico	feature/versionado-semantico	#30	2	Aprobado y mezclado el 19-sep. El issue se cerró solo y se creó el tag v1.1.0.
#32 Documentar la Definición de terminado	feature/definicion-de-terminado	#48	1	Aprobado por PedroRubioo y mezclado el 20-sep. El issue se cerró solo.
#49 Actualizar el calendario de sprints del README	fix/calendario-sprints	#50	1	Aprobado por PedroRubioo y mezclado el 28-sep. El issue se cerró solo.

La rama mvvm y sus cinco commits son del 4 de septiembre, anteriores a la creación del issue #29 y a la convención feature/*, por lo que esos commits no citan el issue; el trabajo quedó enlazado al integrarse mediante el Pull Request #28, que declara Closes #29. Desde el 16 de septiembre las ramas siguen la convención feature/* o fix/*. Los commits de #14 y #32 citan su issue con Refs #N y, en todos los casos, el enlace que cierra el issue es el Closes #N del Pull Request, como en #50.

Figura 8

Pull Request enlazado con el issue que resuelve

refactor(mvvm): arquitectura MVVM según guía oficial de Flutter (Fases 0-2) + demanda no atendida #28

Merged Ennimex merged 5 commits into main from mvvm 6 hours ago

Conversation 1 Commits 5 Checks 0 Files changed 100 +1,935 -751

Ennimex commented yesterday • edited by PedroRubio

Descripción

Migra la app a MVVM siguiendo la guía oficial de Flutter (docs.flutter.dev/app-architecture):
 Fase 0 reorganiza lib/ en ui/ • domain/ • data/ • config/ • utils/ • routing/ con imports absolutos; Fase 1 agrega la interfaz ApiClient y un FakeApiClient para pruebas sin red; Fase 2 crea 14 repositorios y redirige 19 pantallas + 6 providers, que ya no importan ApiService / ApiConstants . Incluye además el commit previo de demanda no atendida (búsquedas y "Avísame" en agotados) y FAQ/Nosotros/Legales leídos desde el panel. Cero cambio visual por diseño: los repositorios devuelven la respuesta cruda del backend; el tipado llega con los ViewModels (Fase 3). Detalle completo en REPORTE_MVVM_APP.docx .

Issue relacionado

Closes #29

Relacionados: avanza #15 (retira la prueba de plantilla del contador; faltan app_test.dart y el helper) y habilita #16 y #17 (el FakeApiClient permite probar providers y pantallas sin red).

Tipo de cambio

- ☒ feat — nueva funcionalidad
- ☐ fix — corrección de error
- ☒ docs — documentación
- ☒ chore / build / ci — configuración, firma, versionado o pipeline
- ☒ test — pruebas
- ☒ refactor — cambio interno sin alterar comportamiento

Reviewers

PedroRubio ✓

Assignees

Ennimex

Labels

calidad

Projects

None yet

Milestone

Sprint 1 — Habilitar el pipeline

Development

Successfully merging this pull request may close these issues.

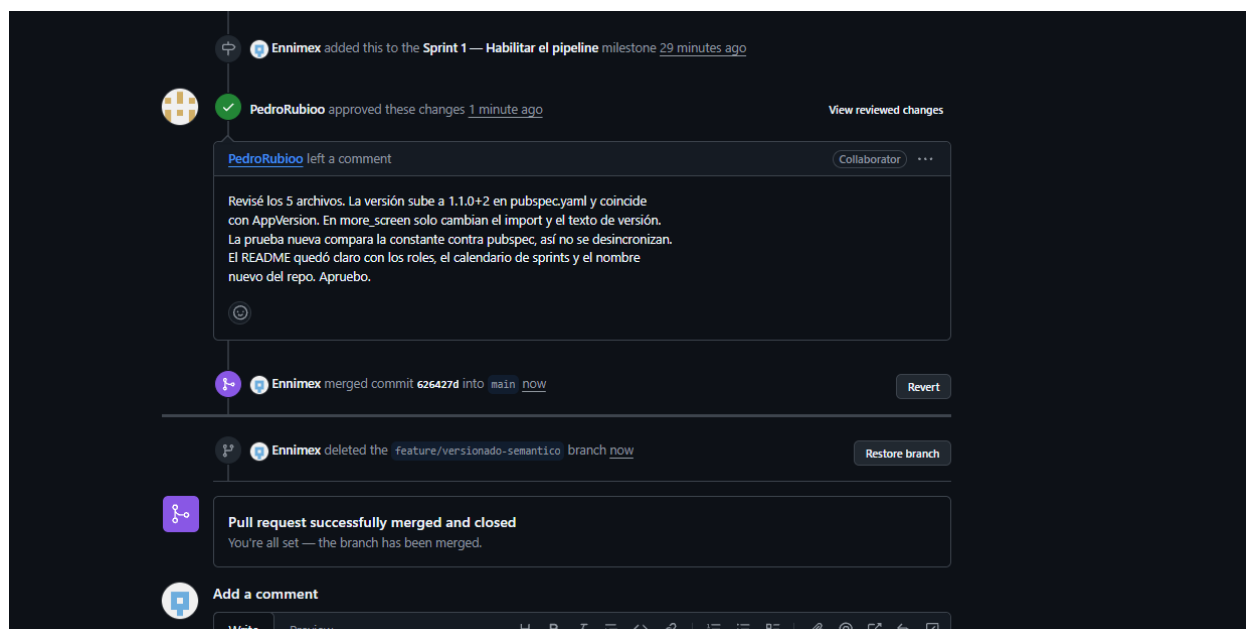
Adoptar arquitectura MVVM en la app (Fases 0-2)

2 participants

Ennimex, PedroRubio

Figura 9

Pull Request #30 aprobado por el compañero y mezclado a main con el mismo flujo



1.8 Definición de terminado

Es el acuerdo del equipo sobre cuándo una actividad se considera terminada y una funcionalidad, liberable. Es la misma lista para todas las tarjetas del tablero y la revisa el compañero en cada Pull Request. Una historia o una tarea está terminada cuando:

1. Cumple todos sus criterios de aceptación.
2. Su Pull Request hacia main declara Closes #N y fue aprobado por el compañero.
3. flutter analyze no reporta errores ni avisos nuevos respecto a main.
4. flutter test pasa en verde, y el cambio agrega o actualiza pruebas cuando toca lógica.
5. No introduce secretos: sin llaves, tokens, keystores, key.properties ni archivos .env.
6. Se probó en dispositivo físico o emulador, y el Pull Request anota qué se probó.
7. El Pull Request está mezclado, el issue cerrado y la tarjeta en Done.

Una liberación está terminada cuando, además:

8. main lleva un tag SemVer vX.Y.Z que coincide con pubspec.yaml.
9. Existe un APK firmado con la llave de release, generado desde ese tag.

La Definición de terminado está publicada en el README del repositorio (issue #32, Pull Request #48) y en la descripción del tablero, para que quien abra cualquiera de los dos enlaces la encuentre.

2. Configuración del repositorio

2.1 Creación y configuración del repositorio

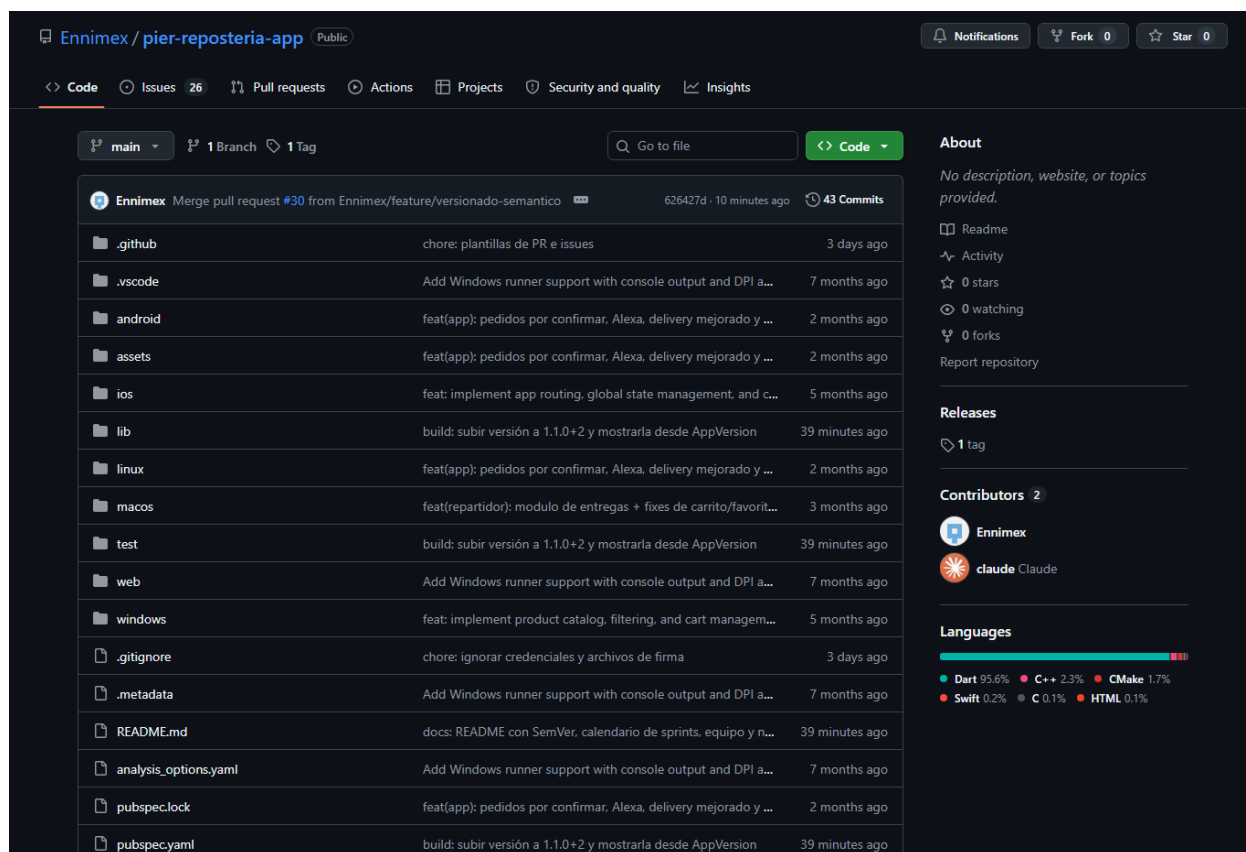
Tabla 13

Datos de creación y configuración del repositorio

Propiedad	Valor
Nombre	pier-reposteria-app
Dirección	https://github.com/Ennimex/pier-reposteria-app
Plataforma	GitHub, cuenta propietaria Ennimex
Visibilidad	Público
Rama principal	main, protegida
Creación	25 de febrero de 2026, al iniciar el desarrollo de la app
Configuración DevOps	16 de septiembre de 2026, para esta actividad

El repositorio ya existía porque la app está en desarrollo desde febrero. Para esta actividad se configuró con enfoque DevOps: README, .gitignore reforzado, plantillas de Pull Request y de issues, seis etiquetas propias (epica, historia, tarea, devops, calidad y seguridad), seis milestones, el tablero de planeación y la regla que protege main.

El 19 de septiembre se renombró de pier_pasteleria a pier-reposteria-app para que haga juego con el repositorio del backend. GitHub redirige la dirección anterior, así que ningún enlace se rompió.

Figura 10*Página principal del repositorio*

2.2 Estructura inicial del proyecto y archivo README

Es un proyecto Flutter. La carpeta `lib/` sigue la arquitectura MVVM de la guía oficial de Flutter (Flutter, s.f.), que separa interfaz, dominio y datos:

```
lib/
  config/      constantes de la API, datos del negocio, versión
  data/        services (cliente HTTP) y repositories (uno por dominio)
  domain/      modelos: producto, pedido, usuario...
  routing/     rutas con go_router
  ui/          pantallas por función y estado global
  utils/       validadores, formateadores, logger
test/         pruebas sin red sobre un cliente de API falso
android/ ios/ proyectos nativos
.github/      plantillas de Pull Request y de issues
pubspec.yaml  dependencias y versión de la app
README.md
```

El README explica qué es la app y documenta: equipo y roles, stack, estructura de carpetas, requisitos, cómo correr el proyecto, estrategia de ramas, convención de commits, versionado, calendario de sprints y enlaces al tablero y al backend.

Figura 11

Archivo README del proyecto

Preview

Code

Blame

184 líneas (147 loc) · 7.77 KB

Raw

Pier Repostería — App móvil

Aplicación móvil (Android / iOS) de **Pier Repostería**, una pastelería con venta en línea. Permite a los clientes explorar el catálogo, guardar favoritos, armar su carrito, pagar con tarjeta, dar seguimiento a sus pedidos, dejar reseñas y solicitar reembolsos o levantar quejas. Incluye además un **módulo de repartidor** para gestionar entregas y una **vinculación con Alexa** para consultar pedidos por voz.

La app consume el backend de Pier (Node + Express) desplegado en Render; no contiene lógica de negocio del lado del servidor.

Equipo y roles

Integrante	Rol (Scrumban)	Responsabilidades
Pedro (@PedroRubioo)	Product Owner · DevOps / Release Engineer	Define y prioriza las épicas del backlog. Ambientes, firma y publicación de liberaciones, pipeline de CI/CD y monitoreo. Revisa y aprueba los Pull Requests hacia <code>main</code> . Responsable del backend (API) que consume la app.
Alexander (@Ennimex)	Desarrollo móvil · QA · Documentación técnica	Implementa las historias de la app en Flutter y la arquitectura MVVM. Pruebas unitarias, de widget y de integración. Versionado, README y documentación técnica.

El responsable de cada actividad aparece como *Assignee* en su issue y en el [tablero de planeación](#).

Stack

Capa	Tecnología
Framework	Flutter (Dart 3.x)
Estado	<code>provider</code>
Navegación	<code>go_router</code>
HTTP	<code>http</code>
Pagos	<code>flutter_stripe</code> (Stripe Payment Sheet)
Autenticación	JWT propio + <code>google_sign_in</code> (OAuth de Google)
Sesión local	<code>shared_preferences</code>
Backend	Node + Express + PostgreSQL, desplegado en Render

2.3 Configuración de .gitignore para Flutter

Se parte de la plantilla oficial de Flutter y se añade un bloque propio para que nunca se suban credenciales ni archivos de firma.

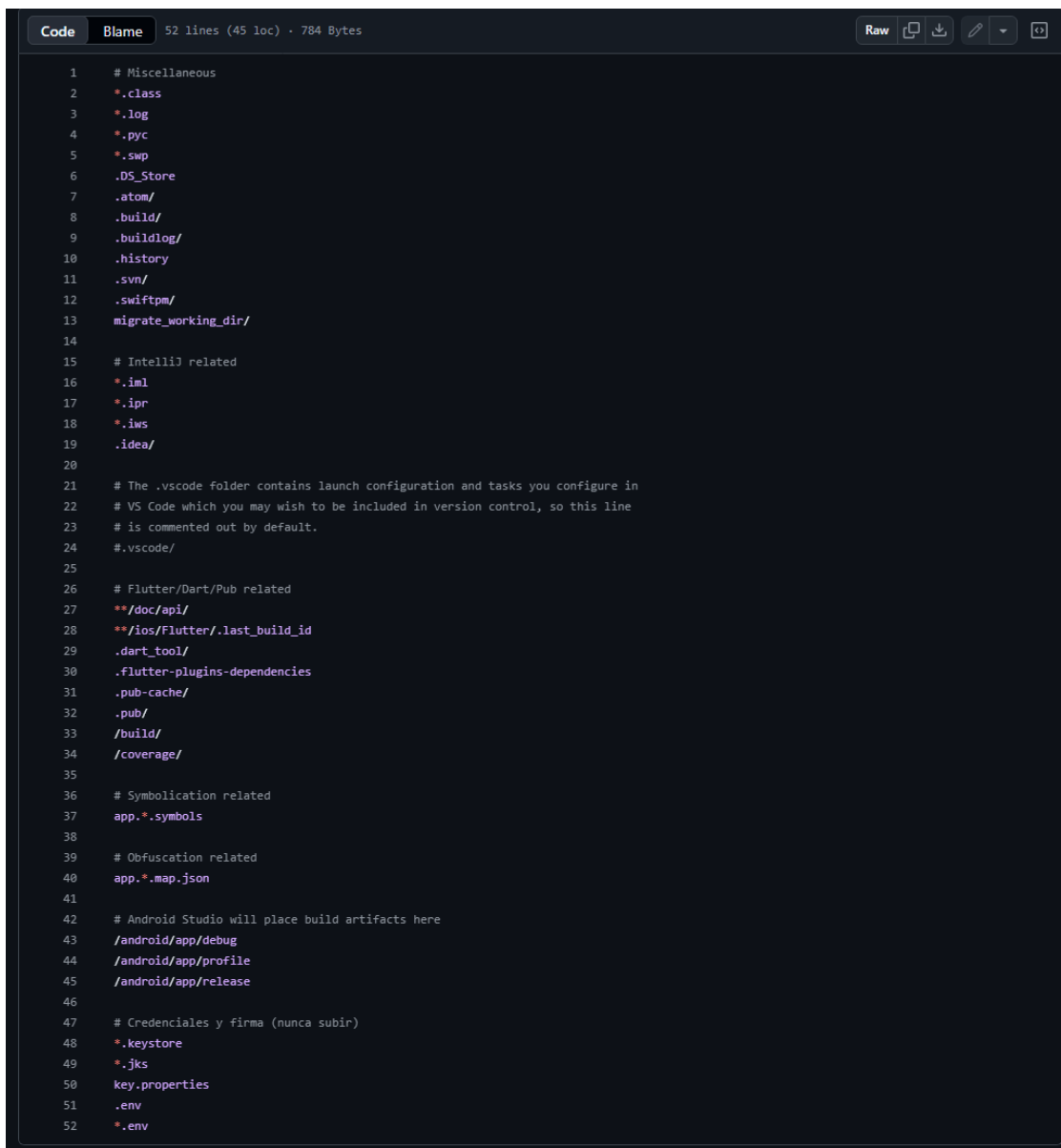
Tabla 14

Patrones del archivo .gitignore y motivo de cada grupo

Grupo	Patrones	Motivo
Flutter, Dart y Pub	.dart_tool/, .pub-cache/, .pub/, /build/, /coverage/, .flutter-plugins-dependencies	Archivos generados que se reconstruyen con flutter pub get.
Artefactos de Android	/android/app/debug, /android/app/profile, /android/app/release	Salidas de compilación del APK.
Símbolos y ofuscación	app.*.symbols, app.*.map.json	Se generan en cada build de release.
Entornos de desarrollo	Archivos de IntelliJ y Android Studio	Configuración local de cada máquina.
Credenciales y firma	*.keystore, *.jks, key.properties, .env, *.env	Secretos. Si se suben, cualquiera podría firmar la app o usar las llaves.

Figura 12

Archivo .gitignore configurado para Flutter



```
Code Blame 52 lines (45 loc) · 784 Bytes
1  # Miscellaneous
2  *.class
3  *.log
4  *.pyc
5  *.swp
6  .DS_Store
7  .atom/
8  .build/
9  .buildlog/
10 .history
11 .svn/
12 .swiftpm/
13 migrate_working_dir/
14
15 # IntelliJ related
16 *.iml
17 *.ipr
18 *.iws
19 .idea/
20
21 # The .vscode folder contains launch configuration and tasks you configure in
22 # VS Code which you may wish to be included in version control, so this line
23 # is commented out by default.
24 #.vscode/
25
26 # Flutter/Dart/Pub related
27 **/doc/api/
28 **/ios/Flutter/.last_build_id
29 .dart_tool/
30 .flutter-plugins-dependencies
31 .pub-cache/
32 .pub/
33 /build/
34 /coverage/
35
36 # Symbolication related
37 app.*.symbols
38
39 # Obfuscation related
40 app.*.map.json
41
42 # Android Studio will place build artifacts here
43 /android/app/debug
44 /android/app/profile
45 /android/app/release
46
47 # Credenciales y firma (nunca subir)
48 *.keystore
49 *.jks
50 key.properties
51 .env
52 *.env
```

2.4 Incorporación de los integrantes como colaboradores

Tabla 15

Cuentas con acceso al repositorio y al tablero

Cuenta	Integrante	Repositorio	Tablero
Ennimex	Alexander Hernandez Meza	Propietario (Admin)	Propietario
PedroRubioo	Pedro Rubio Angeles	Colaborador (Write)	Admin
afelipe23	Ing. Ana María Felipe Redondo	Lectura, es público	Read

Figura 13

Colaboradores del repositorio

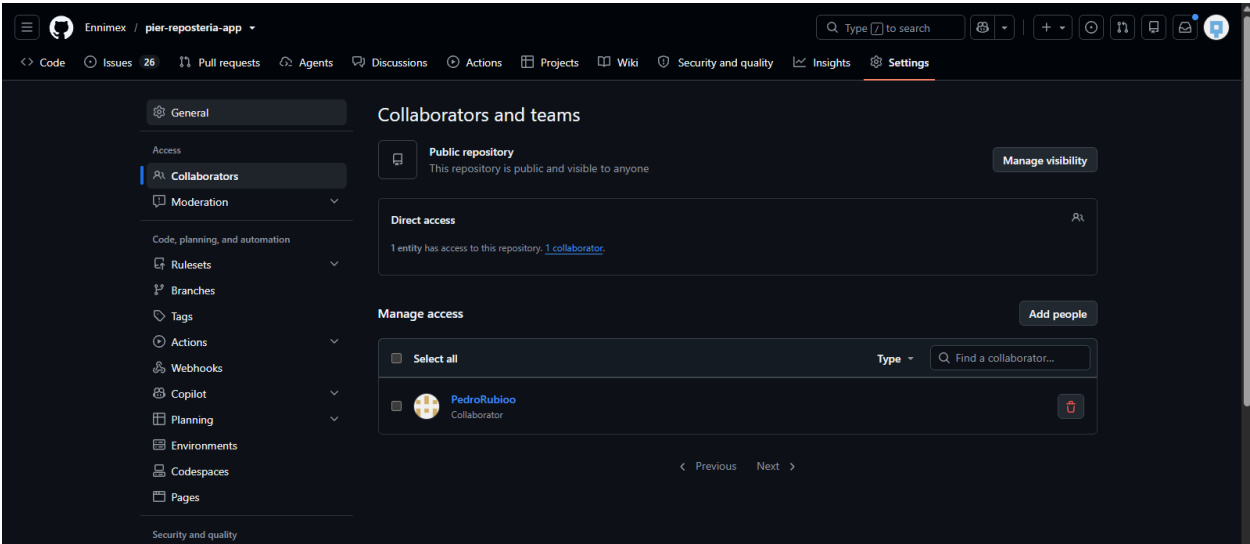
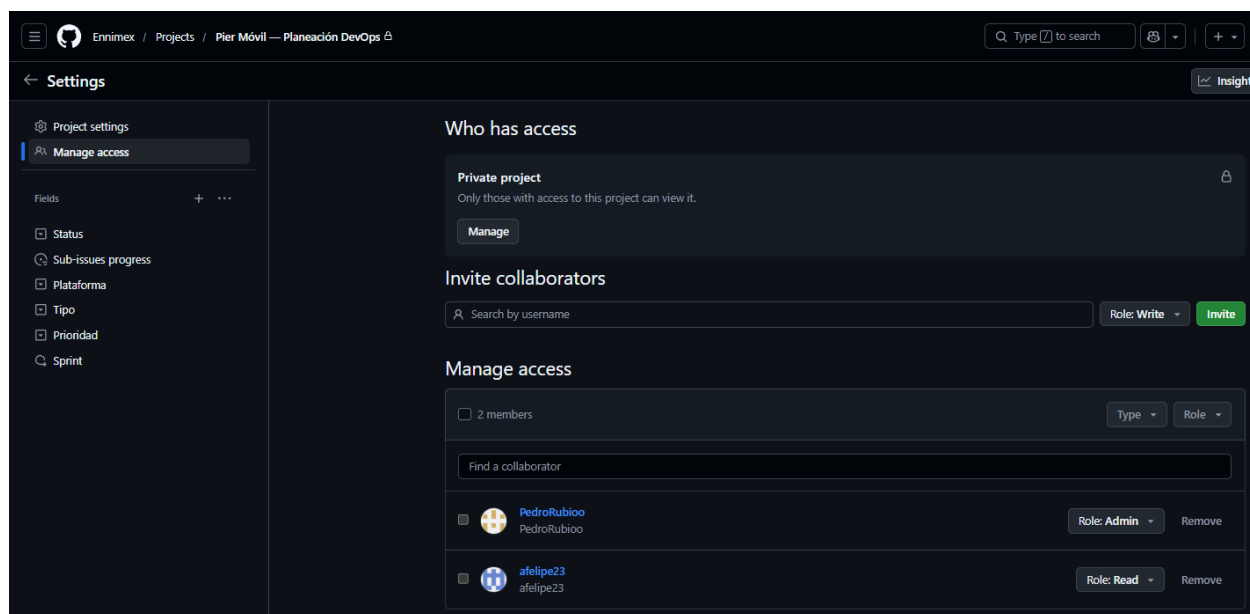


Figura 14*Acceso al tablero de planeación*

2.5 Estrategia de ramas para el desarrollo (GitHub Flow)

- `main` es la única rama de larga vida y siempre debe poder liberarse. No existe rama `develop`.
- Todo cambio nace en una rama `feature/<descripcion>` creada desde `main`. Para correcciones se usa `fix/<descripcion>`.
- La rama se integra a `main` únicamente mediante Pull Request, con al menos una aprobación del compañero.
- Cada Pull Request enlaza el issue que resuelve con `Closes #N`.
- Al mezclar, la rama se borra. La siguiente actividad abre una rama nueva desde `main` actualizado.

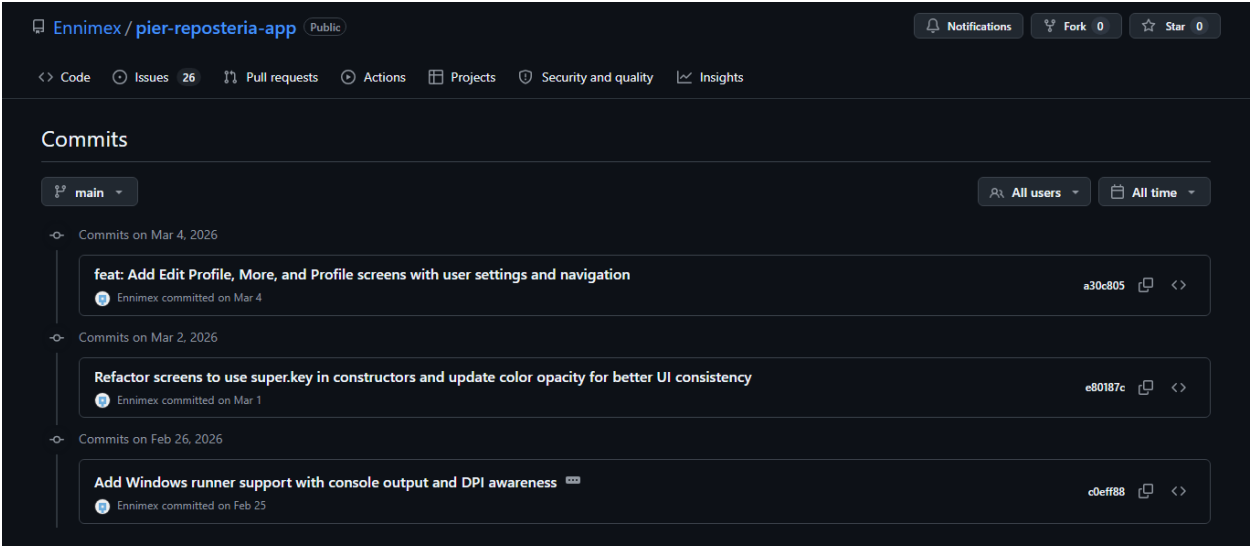
Los mensajes de commit siguen Conventional Commits (Conventional Commits, s.f.): `<tipo>(<ámbito>): <descripción>`, con los tipos `feat`, `fix`, `docs`, `chore`, `ci`, `test`, `refactor` y `build`. La estrategia completa está documentada en el README.

2.6 Primer commit del proyecto

El primer commit es c0eff88, del 25 de febrero de 2026, hecho por Ennimex. Con él quedó almacenado el proyecto base de Flutter. Desde entonces main acumula más de 40 commits con todo el código fuente de la app.

Para comprobar que el código está correctamente almacenado basta con clonarlo y ejecutarlo: `git clone`, `flutter pub get` y `flutter run`. `flutter test` pasa en verde sobre main y `flutter analyze` no reporta errores de compilación; los 37 avisos heredados (1 warning y 36 informativos) se atienden en la actividad #18 del Sprint 1.

Figura 15
Historial de commits con el primer commit del proyecto



2.7 Reglas para integrar código mediante Pull Requests

La rama main está protegida con un conjunto de reglas (ruleset) llamado «Proteger main» (GitHub, s.f.-c), activo y sin lista de excepciones. Aplica a todos, incluido el propietario del repositorio.

Tabla 16
Reglas del conjunto de reglas «Proteger main»

Regla	Configuración
Requerir Pull Request	Ningún commit entra a main por push directo.

Regla	Configuración
Aprobaciones requeridas	1, de una persona distinta al autor.
Descartar aprobaciones viejas	Si el autor sube commits nuevos, la aprobación anterior se anula.
Bloquear force push	No se puede reescribir el historial de main.
Restringir borrado	No se puede eliminar la rama main.

Además, la plantilla de Pull Request obliga a declarar el issue relacionado, el tipo de cambio y un checklist: rama correcta, commits convencionales, analyze y test en verde, sin secretos, probado en dispositivo y revisado por el compañero.

Figura 16

Regla de protección de la rama main

Rulesets / Proteger main Active

Name Proteger main

Enforcement status **Active** - This ruleset will be enforced

Target branches

Branch targeting criteria

+ Default

Rules
Which rules should be applied?

Branch rules

Restrict deletions
Only allow users with bypass permissions to delete matching refs.

Require a pull request before merging
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Show additional settings ▾

Block force pushes
Prevent users with push access from force pushing to refs.

La regla se probó con el Pull Request #28, que pasó por los tres estados:

Figura 17

Pull Request bloqueado mientras no tiene aprobación

refactor(mvvm): arquitectura MVVM según guía oficial de Flutter (Fases 0-2) + demanda no atendida #28

Ennimex wants to merge 5 commits into main from mvvm

Conversation 0 Commits 5 Checks 0 Files changed 100

Ennimex commented 10 hours ago · edited

Descripción

Migra la app a MVVM siguiendo la guía oficial de Flutter (docs.flutter.dev/app-architecture):
 Fase 0 reorganiza lib/ en ui/ · domain/ · data/ · config/ · utils/ · routing/ con imports absolutos; Fase 1 agrega la interfaz `ApiClient` y un `FakeApiClient` para pruebas sin red; Fase 2 crea 14 repositorios y rediseña 19 pantallas + 6 providers, que ya no importan `ApiService` / `ApiConstants`. Incluye además el commit previo de demanda no atendida (búsquedas y "Avísame" en agotados) y FAQ/Nosotros/Legales leídos desde el panel. Cero cambio visual por diseño: los repositorios devuelven la respuesta cruda del backend; el tipado llega con los ViewModels (Fase 3). Detalle completo en [REPORTE_MVVM_APP.docx](#).

Issue relacionado

Closes #29

Relacionados: avanza #15 (retira la prueba de plantilla del contador; faltan `app_test.dart` y el helper) y habilita #16 y #17 (el `FakeApiClient` permite probar providers y pantallas sin red).

Tipo de cambio

- ☒ feat — nueva funcionalidad
- ☐ fix — corrección de error
- ☒ docs — documentación
- ☒ chore / build / ci — configuración, firma, versionado o pipeline
- ☒ test — pruebas
- ☒ refactor — cambio interno sin alterar comportamiento

Checklist

- ☐ La rama sale de main y sigue el patrón `feature/*` (o `fix/*`) — sale de main (rebasada sobre #571, #64) pero se llama mvvm; se creó antes de definir el patrón
- ☒ Los commits siguen Conventional Commits
- ☒ Flutter analyze pasa sin errores nuevos (37 issues, mismo piso que main)

Reviewers: PedroRubio

At least 1 approving review is required to merge this pull request.

Still in progress? [Convert to draft](#)

Assignees: Ennimex

Labels: calidad

Projects: None yet

Milestone: Sprint 1 — Habilitar el pipeline

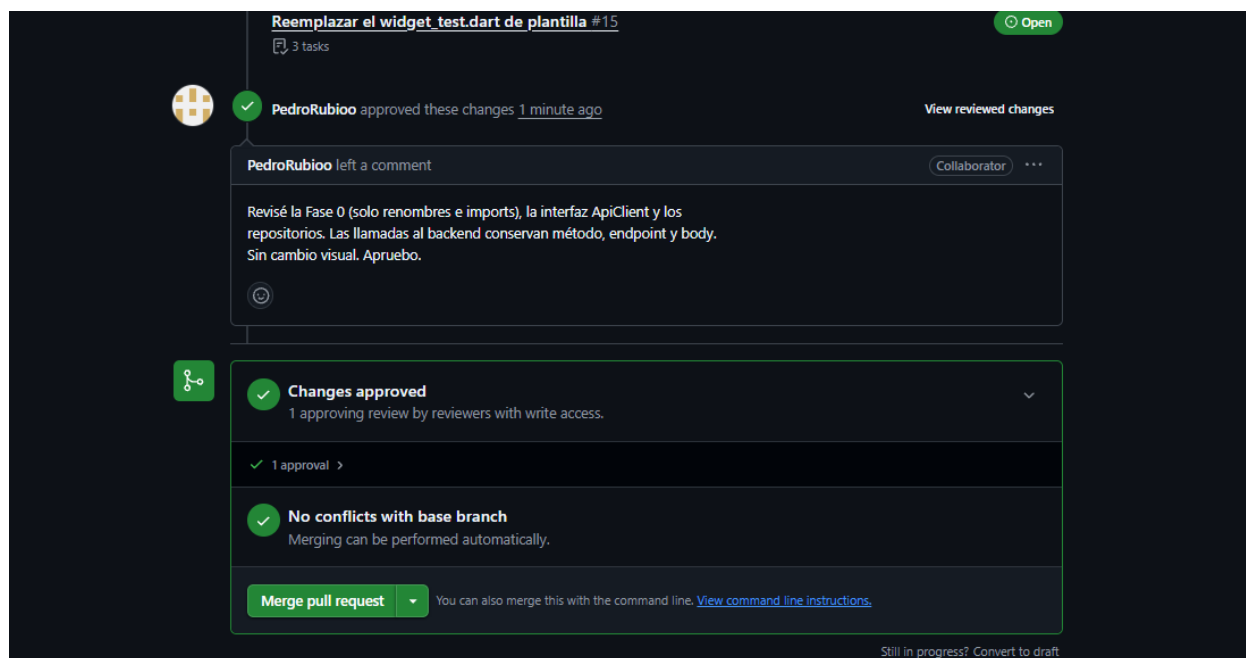
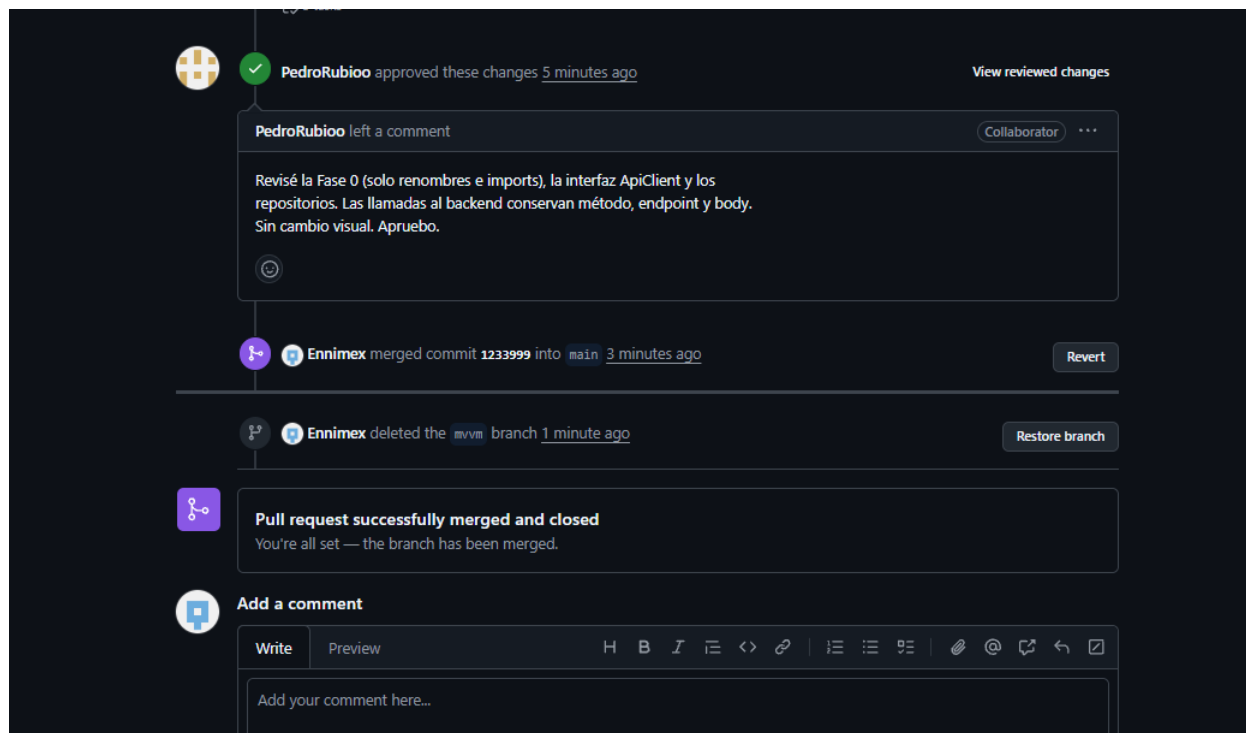
Development: Successfully merging this pull request may close these issues.

Adoptar arquitectura MVVM en la app (Fases 0-2)

Notifications: Unsubscribe

You're receiving notifications because you authored the thread.

1 participant

Figura 18*Pull Request aprobado por el revisor***Figura 19***Pull Request mezclado a main*

3. Conclusión

La planeación y el repositorio quedaron conectados: cada actividad del tablero es un issue con responsable, sprint y estado, y solo puede llegar a main por un Pull Request revisado que la enlaza y que cumple la Definición de terminado del equipo. Con esa base, los siguientes sprints agregan la parte automatizada del enfoque DevOps: pruebas e integración continua con GitHub Actions, análisis de calidad con SonarCloud, publicación del APK en GitHub Releases y monitoreo con Sentry.

Referencias

- Conventional Commits. (s.f.). *Conventional Commits 1.0.0*.
<https://www.conventionalcommits.org/es/v1.0.0/>
- Flutter. (s.f.). *Guide to app architecture*. <https://docs.flutter.dev/app-architecture/guide>
- GitHub. (s.f.-a). *Acerca de Projects*. GitHub Docs. <https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>
- GitHub. (s.f.-b). *Flujo de trabajo de GitHub*. GitHub Docs. <https://docs.github.com/es/get-started/using-github/github-flow>
- GitHub. (s.f.-c). *Acerca de los conjuntos de reglas*. GitHub Docs.
<https://docs.github.com/es/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets>
- Ladas, C. (2009). *Scrumban: Essays on kanban systems for lean software development*. Modus Cooperandi Press.
- Preston-Werner, T. (s.f.). *Versionado semántico 2.0.0*. <https://semver.org/lang/es/>